

CS 305: THEORY OF COMPUTATION — WEEK 1 LECTURE

Preliminaries, Formal Languages & Models of Computation

Mathematical Foundations, The Chomsky Hierarchy, and Modern AI Frontiers

School of Technology

Ghana Institute of Management and Public Administration (GIMPA)

Textbook Reference: Michael Sipser, *Introduction to the Theory of Computation* (3rd Ed.), Chapter 0

Course Architecture: What is Theory of Computation?

From fundamental mathematical limits to modern algorithmic efficiency

1. Automata Theory

Finite & Stack Machines

- Memory-bounded computational models.
- Deterministic & Nondeterministic Finite Automata (DFA/NFA).
- Context-Free Grammars & Pushdown Automata (PDA).
- Applied in: Compilers, Regex, and Deep Packet Inspection.

2. Computability Theory

1930s – 1950s: Solvability

- What is computable in principle?
- Turing Machines (Alan Turing, 1936).
- The Church–Turing Thesis.
- Undecidability & Halting Problem (A_{TM}).
- Limits of static program verification (Rice's Theorem).

3. Complexity Theory

1960s – Present: Efficiency

- What is computable in practice?
- Asymptotic Time & Space complexity bounds.
- Classes **P**, **NP**, and **NP-Complete**.
- Cook–Levin Theorem & 3SAT.
- Modern Cryptography, SAT solvers, and Quantum (**BQP**).

Central Question: What are the fundamental capabilities and limitations of digital computers, and how does mathematical theory guide modern software systems?

Course Mechanics & Evaluation Criteria

Pedagogical structure, grading breakdown, and learning expectations

Course Structure & Assessment

Grading Distribution

- **Problem Sets (4 Assignments, 25%):** Rigorous mathematical proofs and automata design.
- **Quizzes & Checks (2 Quizzes, 15%):** In-class conceptual checkpoints.
- **Midterm Examination (20%):** Automata, Regular Languages & CFLs.
- **Term Project (15%):** Hands-on automata simulator or formal verification tool.
- **Final Examination (25%):** Comprehensive assessment.

Primary Literature & Standards

Required Textbook

Michael Sipser, *Introduction to the Theory of Computation*, 3rd International Edition, Cengage Learning.

Academic Expectations

- **Mathematical Rigor:** Exact definitions, precise notation, and formal proofs.
- **Collaboration Policy:** Discussion encouraged; all proofs and code written individually.

Diagnostic Prerequisite: This course assumes familiarity with discrete mathematics: set theory, relations, functions, graphs, and proof by induction/contradiction. Review Sipser Chapter 0 this week.

Why Theory of Computation in 2026?

Theoretical boundaries govern everyday production engineering

Everyday Systems Engineered on Theory

Compilers & Lexical Analyzers

Lexers (Flex, LLVM) convert source code to tokens in guaranteed $\mathcal{O}(n)$ time by compiling regex to DFAs.

Network Firewalls (Zeek / Snort)

Deep Packet Inspection engines scan gigabit-per-second traffic in real-time using parallel finite state machines.

Modern Frontiers: AI, Security & Blockchain

Large Language Models (LLMs)

Grammar-guided decoding (Outlines/Guidance) forces LLM logits through CFG state machines to guarantee valid JSON/SQL.

Smart Contract Gas (Ethereum)

Because the Halting Problem is undecidable, blockchains cannot statically check termination; they charge “Gas” per step.

Engineering Takeaway: Theory is not abstract detachment; it is the ultimate architectural tool that tells engineers what is mathematically possible, what is intractable, and what is strictly impossible.

Mathematical Foundations: Alphabets (Σ)

The atomic building blocks of all formal languages

Formal Definition

Definition 1.1: Alphabet

An **alphabet** Σ is a non-empty, **finite set** of symbols.

Critical Mathematical Requirements:

- $\Sigma \neq \emptyset$ (Must contain at least one symbol).
- $|\Sigma| < \infty$ (Must be finite; infinite alphabets are excluded in standard automata theory).
- Symbols are atomic primitives.

Standard Computing Alphabets

Ubiquitous Computing Examples:

- **Binary Alphabet:** $\Sigma = \{0, 1\}$ (Digital circuits, assembly).
- **Decimal Alphabet:** $\Sigma_{10} = \{0, 1, 2, \dots, 9\}$.
- **English Alphabet:** $\Sigma_{\text{ENG}} = \{a, b, c, \dots, z\}$.
- **DNA Genetic Alphabet:** $\Sigma_{\text{DNA}} = \{A, C, G, T\}$.
- **ASCII Byte Alphabet:**
 $\Sigma_{256} = \{0x00, \dots, 0xFF\}$.

Common Student Mistake: An alphabet Σ is a set of *symbols*, NOT a set of strings. For example, $\{00, 11\}$ is a language over $\{0, 1\}$, not an atomic alphabet.

Strings (Words) Over an Alphabet

Finite sequences formed from alphabet symbols

Formal Definitions

Definition 1.2: String (Word)

A **string** w over alphabet Σ is a **finite sequence** of symbols from Σ :

$$w = w_1 w_2 \dots w_n \quad (w_i \in \Sigma)$$

String Length ($|w|$)

The number of symbols in w . If $w = 10110$, $|w| = 5$.

The Empty String (ε or λ)

The unique string containing zero symbols: $|\varepsilon| = 0$.

String Concatenation & Algebra

Concatenation ($u \cdot v$ or uv)

If $u = a_1 \dots a_m$ and $v = b_1 \dots b_n$, then:

$$uv = a_1 \dots a_m b_1 \dots b_n$$

Algebraic Laws:

- **Length Additivity:** $|uv| = |u| + |v|$.
- **Identity Element:** $\varepsilon u = u\varepsilon = u$.
- **Associativity:** $(uv)w = u(vw)$.
- **Non-Commutative:** In general, $uv \neq vu$ ($01 \neq 10$).

Algebraic Insight: The set of strings over Σ with concatenation form a **monoid** with identity element ε .

String Operations: Powers, Reversal, Substrings

Essential mathematical operations utilized throughout automata proofs

String Powers (w^k)

Inductive Definition

$$w^0 = \varepsilon$$

$$w^k = w^{k-1}w \quad (k \geq 1)$$

Examples:

- $(01)^3 = 010101$
- $a^4 = aaaa$
- $(11)^0 = \varepsilon$

String Reversal (w^R)

Inversion of Symbols

If $w = w_1 w_2 \dots w_n$:

$$w^R = w_n \dots w_2 w_1$$

Key Inversion Law:

$$(uv)^R = v^R u^R$$

Palindrome Definition: w is a palindrome iff $w = w^R$.

Substrings & Factors

Prefix, Suffix, Substring

Let w be a string:

- **Prefix:** x where $w = xy$.
- **Suffix:** y where $w = xy$.
- **Substring:** y where $w = xyz$.

Note: ε is a prefix, suffix, and substring of every string.

Proof Technique Tip: Proving $(uv)^R = v^R u^R$ is done by induction on $|v|$. Notice how string concatenation reverses order, identical to matrix transposition $(AB)^T = B^T A^T$!

Formal Languages Over an Alphabet

A language is a set of strings: framing computation as decision problems

Kleene Star & The String Universe

The Kleene Star (Σ^*)

The set of **all finite strings** over Σ , including ε :

$$\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k = \{\varepsilon, 0, 1, 00, 01, 10, 11, \dots\}$$

Kleene Plus (Σ^+)

All strings except the empty string:

$$\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$$

Formal Definition of a Language

Definition 1.3: Formal Language

A **language** L over alphabet Σ is any subset of Σ^* :

$$L \subseteq \Sigma^*$$

Examples over $\Sigma = \{0, 1\}$:

- $L_1 = \{w \mid w \text{ contains substring } 11\}$
- $L_2 = \{w \mid w \text{ has an even number of 0s}\}$
- $L_3 = \{0^n 1^n \mid n \geq 0\} = \{\varepsilon, 01, 0011, \dots\}$
- $L_{\text{prime}} = \{1^p \mid p \text{ is prime}\}$

Critical Distinction: $\emptyset$ vs. $\{\varepsilon\}$ vs. ε . $\emptyset$ is the **empty language** ($|\emptyset| = 0$ strings). $\{\varepsilon\}$ is a **language containing 1 string** ($|\{\varepsilon\}| = 1$). ε is a **string** (not a set). Never confuse $\emptyset$ and $\{\varepsilon\}$!

In-Class Check-in 1.1: Mathematical Foundations

Test your conceptual understanding of alphabets, strings, and languages

Conceptual Questions

Check-in Problem 1.1A

Let $\Sigma = \{a, b\}$.

1. What is $|\Sigma^3|$?
2. Is $\emptyset \subseteq \Sigma^*$?
3. Is $\varepsilon \in \emptyset$?
4. Is $\varepsilon \in \Sigma^*$?

Check-in Problem 1.1B

True or False: If language L is infinite, must the underlying alphabet Σ be infinite?

In-Class Workspace & Discussion

Student Analysis Prompts:

- **Cartesian Power Rule:** How does $|\Sigma^k|$ scale with alphabet size $|\Sigma|$ and word length k ?
- **Set-Theoretic Distinction:** Recall the strict difference between the *empty set* $\emptyset$ (containing zero strings) and the *empty string* ε (containing zero symbols).
- **Countability Principle:** Review Sipser Section 0.2 on countably infinite languages Σ^* constructed over finite alphabets Σ .
- **Collaborative Action:** Discuss your deductions with your peer before open class review.

[Active learning drill — solutions analyzed interactively in lecture.]

Pedagogical Key: Every computational decision problem can be formalized as a **language decision problem**: "Given string $w \in \Sigma^*$, is $w \in L$?"

Modern AI Connection: LLM Tokenization Alphabets

Byte-Pair Encoding (BPE) and why token alphabets break character reasoning

Subword Tokenization (BPE)

The Token Vocabulary Alphabet (Σ_V)

LLMs define an alphabet of subword tokens:

$$\Sigma_V = \{0, 1, 2, \dots, |V| - 1\}$$

where $|V| \approx 32,000$ to $128,000$ unique tokens.

Tokenization Mapping:

$$\text{tokenize} : \Sigma_{\text{UTF8}}^* \rightarrow \Sigma_V^*$$

Example (cl100k_base):

“strawberry” $\rightarrow$ [“str”, “aw”, “berry”]

= [4999, 1378, 19233] (3 symbols in Σ_V^*).

The “How Many r’s?” Mystery

Why LLMs Fail Character Counts

Observation: When asked “*How many r’s in strawberry?*”, unassisted LLMs often say 2 instead of 3.

Theoretical Cause:

- Input sequence is 3 tokens long.
- The letter ‘r’ is NOT an atomic symbol in Σ_V .
- Model must recall spelling from dense vector weights without string inspection.

Formal Language Lesson: The choice of alphabet Σ determines the expressivity and difficulty of all downstream computational tasks.

The Chomsky Hierarchy: 4 Classes of Expressivity

Noam Chomsky (1956): Classifying grammars and their computational automata

Type	Language Class	Recognizing Machine	Memory Model	Production Rules ($A \rightarrow \alpha$)
Type 3	Regular	Finite Automata (DFA/NFA)	$\mathcal{O}(1)$ Constant Memory	$A \rightarrow aB$ or $A \rightarrow a$
Type 2	Context-Free	Pushdown Automata (PDA)	LIFO Stack Memory	$A \rightarrow \gamma$ ($A \in V$)
Type 1	Context-Sensitive	Linear Bounded Automata (LBA)	$\mathcal{O}(n)$ Bounded Tape	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type 0	Turing-Recognizable	Turing Machine (TM)	$\mathcal{O}(\infty)$ Unbounded Tape	$\alpha \rightarrow \beta$ (Unrestricted)

Strict Subset Hierarchy

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0$$

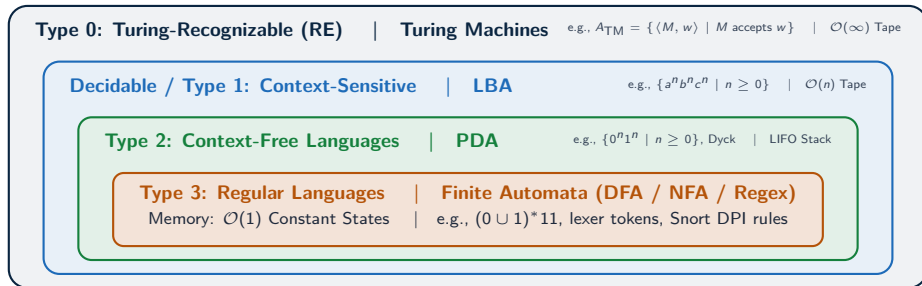
Every regular language is context-free, but CFLs describe nested recursive syntax beyond finite automata.

Why Memory Dictates Power

The progression from Type 3 to Type 0 is an evolution of **storage**: No memory $\longrightarrow$ Stack $\longrightarrow$ Linear tape $\longrightarrow$ Infinite tape.

Visualizing the Expressive Hierarchy

Concentric containment of formal language classes and automata models



Course Trajectory: Over the next 14 weeks, we climb this hierarchy from inside out: Weeks 1–5 (Regular) $\rightarrow$ Weeks 6–8 (Context-Free) $\rightarrow$ Weeks 9–12 (Turing-Recognizable & Undecidable) $\rightarrow$ Weeks 13–15 (Complexity).

Research Spotlight: Neural Networks & The Chomsky Hierarchy

G. Delétang et al. (Google DeepMind), ICLR 2023 Spotlight Paper

The Research Question

Where do Deep Learning Models Lie?

Deep neural networks achieve human-like fluencies, but **what is their theoretical computational capacity** on formal languages?

DeepMind tested architectures across Chomsky levels:

- Feedforward Networks (MLP)
- Recurrent Networks (RNN, LSTM)
- Standard Transformers (Self-Attention)
- Neural Networks augmented with Stack / Tape Memory

DeepMind's Empirical Findings

Key Theoretical Limits:

- **Transformers $\approx$ Regular / Weak CFL:**
Fixed-depth Transformers fail to generalize on deeply nested parentheses (Dyck languages) when input length exceeds training window.
- **Recurrent Models (LSTM) $\approx$ Counter Machines:**
Can count one counter ($a^n b^n$), but fail at multiple nested dependencies.
- **External Memory is Strictly Required:** Neural models achieve Type 2 only with explicit stack, and Type 0 only with read/write tape.

Takeaway for AI Practitioners: Without external scratchpads (Chain-of-Thought) or tool calling, Transformer LLMs are strictly memory-bounded and cannot solve arbitrary algorithmic reasoning!

Introduction to Automata: The Intuition of Finite Control

Abstract machines with finite internal states and no external storage

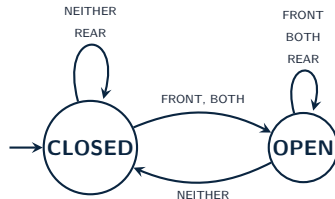
Real-World Paradigm: The Automatic Door

System Specifications

An automatic sliding door at a store entrance:

- **Two States:** CLOSED, OPEN.
- **Alphabet of Sensor Inputs:**
 - ▶ FRONT: Person on front mat.
 - ▶ REAR: Person on rear mat.
 - ▶ BOTH: People on both mats.
 - ▶ NEITHER: No persons detected.

State Transition Graph



Core Intuition of Finite Automata: The machine has **no separate RAM or memory tape**. Its entire history of past inputs is compressed into its **current state**. This makes finite automata extremely fast ($\mathcal{O}(1)$ transition time) but fundamentally limited in counting.

Preview of Week 2: Deterministic Finite Automata

Moving from informal diagrams to the rigorous 5-tuple mathematical definition

The Formal 5-Tuple Definition

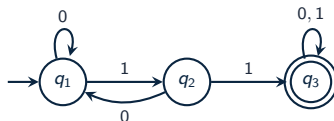
Definition 1.4: DFA (Preview)

A Deterministic Finite Automaton M is a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : Finite set of states.
- Σ : Finite input alphabet.
- $\delta : Q \times \Sigma \rightarrow Q$: Transition function.
- $q_0 \in Q$: Initial (start) state.
- $F \subseteq Q$: Set of accept (final) states.

Example: Substring “11” Recognizer



Language Recognized:

$$L(M) = \{w \in \{0,1\}^* \mid w \text{ contains substring } 11\}$$

Accepts: 01101, 11. Rejects: 01010, 101.

Coming Up in Week 2: Formal execution traces, state transition tables, closure properties (Union/Intersection via Cartesian Product construction), and line-rate Deep Packet Inspection in Zeek/Snort.

Week 1 Synthesis & Action Items

Summary of foundations and preparation for Week 2

Week 1 Concept Checklist

You should now be able to:

- Define alphabet Σ , string w , and language $L \subseteq \Sigma^*$.
- Compute string powers w^k , reversals w^R , and understand length additivity $|uv| = |u| + |v|$.
- Differentiate rigorously between $\emptyset$, $\{\varepsilon\}$, and ε .
- Articulate the 4 levels of the Chomsky Hierarchy and how memory dictates expressive power.
- Explain the connection between LLM subword tokenization and character reasoning limits.

Readings & Diagnostic Assignment

Required Readings

- Sipser (3rd Ed.), **Chapter 0: Introduction** (pp. 1–28).
- Sipser (3rd Ed.), **Section 1.1: Finite Automata** (pp. 31–44).
- Delétang et al., “*Neural Networks and the Chomsky Hierarchy*,” ICLR 2023 (Spotlight).

Action Items

- Complete **Diagnostic Math Review** (Problem Set 0).
- Practice proving statements using induction and contradiction.
- Next Class: **DFA Formal 5-Tuples & State Diagram Design**.

Questions? Office Hours: Tuesdays & Thursdays 14:00–16:00 GMT | GIMPA School of Technology